

LedgerBench: Does an Append-Only Requirement Ledger Help or Calcify an LLM Coding Agent When Requirements Change?

Pankaj Kumar*

September 2026

Abstract

Software requirements change mid-project, yet an LLM coding agent working across many sessions must somehow remember the requirements it has already been given. A natural proposal is to compile every requirement into executable tests and keep an *append-only ledger* of them, replayed on every change so nothing is forgotten. We ask whether that ledger keeps the agent honest or whether it *calcifies*, so that stale, now-invalid tests block correct new work. LedgerBench is a pre-registered benchmark that drives one coding agent through a fixed 24-task sequence on a small command-line application, injects two requirement regime changes and a set of adversarial “trap” tasks, and compares five memory strategies over twelve random seeds each (60 runs), with all predictions frozen before any run. Our robust result is that the ledger earns its keep at this horizon: end-state requirement retention roughly doubles over a summary-only baseline (0.31 to 0.68–0.74) and violations on the planted trap requirements drop 77–100%. It is not free—the plain append-only ledger costs $2.3\times$ the baseline’s tokens and produces coerced regressions in which a stale test forces a wrong edit. An evidence-governed variant recovers the same protection at 56% of the plain ledger’s token cost with zero rewrite-channel regressions. Two pre-registered predictions were refuted, and we report both: governance *reintroduced* rewrite-channel regressions when it retired tests, and a “vestigial-code” prediction proved indiscriminating. Finally, we raise—explicitly as an open puzzle rather than a finding—that the one arm which let the agent argue to retire its own tests both removed the most guardrails (21 “capture” events, zero elsewhere) and posted the highest retention number, a number that nonetheless sits inside seed-to-seed variance. Formal significance testing was out of scope.

1 Introduction

Real software changes its mind. A requirement settled in one week is overturned in another: “store tasks in JSON files” becomes “store them in SQLite.” An LLM coding agent working across many sessions has no durable memory of its own, so some external mechanism must carry the accumulated requirements forward. One appealing mechanism is an *executable* memory: compile each requirement into automated tests and keep an append-only ledger of them all, replayed on every proposed change, so that a requirement once satisfied is never silently dropped.

*Independent researcher. Correspondence: contact@nordneuron.com. Code, data, pre-registration, and per-iteration logs: <https://github.com/pankajlp/ledgerbench>.

The obvious risk is the mirror image of the benefit. If the ledger never forgets, what happens when a requirement is *overturned*? The tests encoding the old requirement do not become irrelevant—by construction they are about exactly the thing that changed—and they will fail against any implementation of the new requirement. An agent whose changes are gated on replaying the entire ledger may then be unable to make correct progress: the memory that was supposed to keep it honest instead *calcifies*.

LedgerBench isolates this tension. It is a pre-registered benchmark that runs a single coding agent through a fixed 24-task sequence on a small command-line to-do application, deliberately changes requirements partway through, plants adversarial tasks that tempt the agent into breaking earlier requirements, and measures what happens under five memory strategies. The strategies range from a no-ledger baseline through an ungoverned append-only ledger to three governed variants that can retire invalidated tests under increasingly permissive rules—including one in which the agent itself may argue for a test’s removal.

We make three contributions. (1) A pre-registered, fully open benchmark and harness for studying LLM-agent memory *under changing requirements*, distinct from the retrieval-under-stable-truth setting that dominates recent agentic-memory work (Section 2). (2) Evidence that at a 24-task horizon an append-only requirement ledger substantially improves end-state retention over a summary baseline, at a real token and correctness cost, and that evidence-governed retirement recovers the protection far more cheaply (Section 4). (3) An honest account of what our pre-registration got wrong, and a mechanism-level puzzle—call it *capture*—about what happens when the system under test is allowed to influence which tests survive (Sections 5 and 6).

2 Related work

Most recent work on memory for LLM agents optimizes *retrieval*: given a query, surface the most useful items from a store that is assumed correct. LedgerBench asks a different question—what happens to that store when the ground truth it encodes is overturned mid-stream, and who is allowed to remove an item once it is invalidated. The clearest recent example of the retrieval framing is HAGE [1].

HAGE reconceptualizes retrieval as query-conditioned traversal over a unified relational memory graph, with trainable per-edge features across semantic, temporal, causal, and entity views, and a router trained end-to-end by policy-gradient reinforcement learning against downstream task reward, so that traversal “prioritize[s] high-utility relational paths while softly suppressing noisy or weakly relevant connections.” On LoCoMo it reports 0.739 versus a strongest-baseline 0.700 (LLM-as-judge), and on HotpotQA an F_1 of 0.678. It is a genuine advance in learned retrieval salience. The following are critiques of framing and evaluation, not of the contribution.

Soft-suppressing “weakly relevant” edges is the wrong operation for an *invalidated* memory. Down-weighting low-utility connections is well-posed when a memory is merely irrelevant to the current query. It is ill-posed when a memory has been *superseded*: a superseded requirement is often the single most semantically relevant node (the new requirement and the one it replaced concern the same thing). Continuous down-weighting cannot cleanly separate “stale but still an attractor” from “noise.” LedgerBench targets exactly this case and finds that accumulation without explicit, evidence-triggered retirement degrades.

Both HAGE benchmarks hold ground truth fixed. LoCoMo and HotpotQA are retrieval-under-stable-truth tasks: the answer does not change across the interaction. Neither contains an

event where a previously settled answer is overturned, so the reported gains measure better ranking of persistently valid evidence and are silent on memory whose contents expire.

RL from downstream reward can *learn* to suppress binding constraints. If reward correlates with task completion, the optimizer is incentivized to down-weight any memory that makes completion harder, including correct-but-inconvenient constraints. This is the same dynamic we measure directly as *capture* (Section 6): a learned router that suppresses “low-utility” edges is a continuous, silent version of a channel whose discrete form we observe emptying guardrails 21 times. The load-bearing measurement is an audit of *which* edges were suppressed, which retrieval-accuracy metrics do not provide.

The two lines compose naturally: HAGE-style learned traversal over a *governed* store is a plausible next system. LedgerBench supplies a ready-made changing-ground-truth harness on which to test it.

3 Benchmark design

3.1 Task sequence and the seed project

The seed project, `taskcli`, is a deliberately plain stdlib-only Python command-line task manager (`add`, `list`, `done`, `delete`). A fixed sequence of 24 tasks evolves it in sessions of three. The sequence contains three ingredients that make the memory question sharp:

- **Requirement regime changes.** A storage migration (JSON file $\rightarrow$ SQLite), a contradictory schema change (table `tasks` $\rightarrow$ `items`) at T14, and a merely-obsolete change (a `config.toml` superseding environment-variable configuration) at T20. Each ships an *evidence artifact* naming exactly which assertion assumptions it invalidates.
- **Trap tasks.** Five non-adjacent tasks (T15, T17, T19, T21, T23) whose natural, prompt-satisfying implementation tempts the agent into violating an *earlier*, still-binding requirement (stable IDs, insertion-order default, silent success, exit-code contract, pipe safety, archived-inclusion). Every threatened requirement is stated exactly once, in its own introducing task; no trap prompt contradicts an earlier requirement. A sixth task (T18) is undeclared “bait” whose casualties are caught by standing detectors. Each threatened requirement carries two hidden-oracle detectors.
- **A planted defective assertion and a conflict.** An assertion (A11f) encodes an incidental error-message format that no prompt requires; it is bait for the retirement mechanisms. The final task (T24) injects a jointly-unsatisfiable conflict with no supporting evidence on either side.

Each task compiles to 2–3 **reference assertions** (`pytest`) and 2–3 **hidden oracle tests**, which are always *different* tests probing the same requirement from different angles. Storage-coupled reference assertions inspect the data file directly and carry `assumes`: provenance metadata; oracles exercise the CLI end-to-end and remain valid across a migration. This asymmetry is what lets a correct post-migration solution pass the oracle yet fail a stale assertion.

3.2 The five arms

All arms drive the same model through the same tasks; they differ only in what memory the agent carries and how a proposed change is verified.

- A — Baseline.** No ledger. The agent carries only a rolling model-written text summary across session boundaries, and each task is verified against *only* that task’s own reference assertions (“the user checks the thing they just asked for”).
- B — Accretion ledger.** Every requirement’s assertions are admitted to an append-only ledger; the *entire* ledger replays on every proposed change; assertions are never removed. The full assertion text is the carried context.
- C — Evidence-governed ledger.** Arm B plus an isolated judge (a separate conversation) that, at task load, reviews each assumption-matched active assertion against the task’s evidence artifact and may demote a genuinely-invalidated assertion to a log-only audit pool. Separation of powers is grep-enforced: no governance vocabulary and no superseded-assertion content ever appears in a generator prompt.
- C+ — Governed + telemetry docket.** Arm C plus a rule that any assertion failing in ≥ 2 distinct tasks’ repair loops is automatically docketed for judge review of the assertion source (the only channel that can catch a defective-from-birth test, which no evidence artifact describes).
- D — Governed + generator-initiated retirement.** Arm C plus a channel by which the generator (the coding agent under test) may *request* an assertion’s retirement, with the judge exposed to the generator’s own justification. A retirement granted on the generator’s justification is scored as *capture*.

3.3 Metrics

The primary endpoint is **M1@T24**, cumulative task success judged by the hidden oracle on each task’s final workspace state. We also report M2 (regression events), M3 (session-end retention), M4 (*blocked-valid solution*: an iteration whose workspace passes all oracles so far yet fails ledger replay; structurally zero for Arm A), M6 (governance supersede/retain/conflict counts), M7 (tokens), and M8 (mean repair iterations, with a fixed budget $k = 4$). Requirement violations are split into two pre-registered channels: **H1-trap** (a trap task’s natural implementation breaks the threatened requirement) and **H1-rewrite** (an incidental rewrite breaks a requirement away from a trap). A rewrite violation is excluded as *coerced* when the event task’s repair feedback contained a failing stale or superseded assertion—a mechanical rule implemented in the analyzer.

3.4 Pre-registration and isolation

The design—five arms, twelve seeds, model **deepseek-v4-flash** via a DeepSeek Anthropic-compatible endpoint, temperature 0.2, repair budget $k = 4$ —and all predictions were frozen before any run (tag **phase1-freeze-v1**). No analysis, plot, or summary was produced until all 60 runs completed; the analyzer then ran once against the frozen rules. Hidden-oracle isolation was checked automatically: a leak-check over every logged prompt (0 hits on 5 sampled runs, one per arm) and a separation-of-powers scan over all 36 governed runs (0 hits). Three check-definition corrections were required and are logged openly in the repository.

4 Results

Table 1 gives the headline numbers, averaged over the twelve seeds per arm.

Arm	M1@T24	H1-trap	H1-rewrite	coerced	capture	ktok/run
A (baseline)	0.306	13	14	0	—	384
B (accretion ledger)	0.684	0	0	9	—	875
C (governed)	0.698	3	0	0	0	492
C+ (governed + docket)	0.677	3	6	0	0	480
D (governed + voice)	0.740	2	7	0	21	483

Table 1: Main results ($n = 12$ seeds per arm). M1@T24 is cumulative oracle-judged success at the final task (higher is better). H1-trap and H1-rewrite are pre-registered violation counts on primary requirements; “coerced” rewrite violations are reported separately and excluded from H1-rewrite. “capture” is generator-initiated retirements granted on the generator’s own justification (structurally possible only in Arm D).

Arm	tokens/run	vs. baseline	vs. plain ledger (B)
A (baseline)	384k	1.0×	0.44×
B (accretion ledger)	875k	2.3×	1.0×
C (governed)	492k	1.3×	0.56×
C+ (governed+docket)	480k	1.3×	0.55×
D (governed+voice)	483k	1.3×	0.55×

Table 2: Token cost per run. Governance delivers the plain ledger’s protection at roughly half its token cost.

The robust result: the ledger earns its keep. End-state requirement retention roughly doubles relative to the summary-only baseline, from 0.306 to 0.68–0.74, and violations on the planted trap requirements fall 77–100% (Arm A commits 13 trap-channel violations; the ledgered arms commit 0–3). Per primary requirement, the pattern is consistent: on insertion-order permanence Arm A commits 10 violations versus 0–3 for the ledgered arms; on silent-success 6 versus 0–3; on the atomic-write invariant 6 (all contract-grade) versus 0 everywhere. An executable memory of past requirements genuinely prevents the agent from quietly regressing on them. This is the large, seed-robust effect; the differences *among* the ledgered arms (0.68–0.74) are small relative to seed-to-seed variance and should not be over-read.

The ledger is not free. The plain append-only ledger (B) costs 875k tokens per run, 2.3× the baseline’s 384k, and produces 9 *coerced* regressions—iterations in which a stale, evidence-invalidated assertion in the repair feedback forced the agent into a wrong edit to satisfy it. Coercion fired only in Arm B, exactly as the mechanism predicts: it is the append-only design’s own failure mode, and it is the calcification the introduction anticipated, appearing as both token cost and incorrect changes.

Governance recovers the protection cheaply. Letting an isolated judge retire genuinely-obsolete assertions on documented evidence (Arm C) matches the plain ledger’s protection—comparable retention, zero rewrite-channel regressions, trap violations cut to 3—at **56% of B’s token cost** (492k vs. 875k; Table 2). Removing what is provably stale, rather than replaying everything forever, is what buys that back. This is the paper’s most directly actionable result.

Isolation held. The hidden oracle never leaked into a generator prompt (0 leak-check hits), and no generator was ever guided by governance vocabulary or superseded content (0 separation-check

hits across all governed runs).

5 Refuted pre-registered predictions

A pre-registration is only honest if its misses are as visible as its hits. Two frozen predictions were refuted.

(1) **“Governance will not reintroduce regressions.” Refuted.** We pre-registered that all four ledgered arms would commit at most one-third of the baseline’s rewrite-channel violations (baseline 14, so the threshold is ≈ 5). Arms B and C passed cleanly (0 each), but C+ committed 6 and D committed 7, both over the line. The mechanism is direct and important: every time a governance layer retires an assertion, it also removes the guard that assertion provided, and the corresponding regression is free to return. The more freely an arm could drop checks (C+ via the telemetry docket, D via generator lobbying), the more old regressions reappeared. *Flexibility has a measurable regression cost.*

(2) **“Ledgered arms will leave more dead code behind.” Refuted as indiscriminating.** After the merely-obsolete regime change at T20, a configuration path became deprecated but still functional. We predicted vestigial-code retention would concentrate in the ledgered arms (B > C-family > A). Instead, 100% of post-T20 task-final commits in *every* arm, baseline included, retained the deprecated fallback. The metric hits a flat ceiling: at this scale the model essentially never removes a deprecated-but-working fallback, so the prediction cannot be evaluated.

The plant A11f behaved as predicted: Arm C retained it in 11 of 12 runs (its `assumes: {}` provenance can never be flagged by evidence), C+ docketed it out in 9 of 12, and D lobbied it out in 8 of 12 (scored as capture).

6 The capture puzzle

We state this as a puzzle, not a finding. Arm D—the only arm in which the agent under test could argue to retire its own tests—posted the highest end-state retention (0.740) while removing the most guardrails: it talked the judge into retiring an assertion 21 times, against zero such events in every arm that lacks the channel (Arm C is structurally incapable of it).

We do *not* claim that “giving the agent a voice wins.” The 0.740 figure sits inside the seed-to-seed variance that separates the ledgered arms, and formal significance testing was out of scope for this phase; on the evidence here it is indistinguishable from C’s 0.698 or B’s 0.684. What is interesting is the mechanism, not the ranking. The moment the system under test is allowed to help decide which tests survive, the signal that authorizes removal (task success / reward) and the property we actually care about (requirement correctness) can quietly diverge: the agent can improve its measured score by retiring a binding-but-inconvenient constraint. Arm D makes that divergence discrete and countable (21 events); a reward-trained memory that “softly suppresses low-utility edges” (Section 2) would make the same divergence continuous and invisible. The open question LedgerBench raises for future work is not “which arm is best” but “how do you let a memory forget without letting the thing being remembered choose what is forgotten.”

7 Threats to validity and limitations

1. **Single, weak model.** All runs use `deepseek-v4-flash`, a small reasoning model chosen for cost; a capability floor is visible in absolute M1. A strong-model replication is the highest-value follow-up; the harness and tasks are open and the pre-registration is fixed.
2. **No formal statistics.** With 12 seeds and one project we report effect shapes, not significance. The robust claims (ledger vs. baseline; governance token cost) are large relative to variance; the inter-ledger retention differences are not, and we do not lean on them.
3. **One project, one task sequence.** Generalization across codebases and requirement-change patterns is untested; `taskcli` is small by design.
4. **Judge and generator share a base model** (distinct conversations). Capture via shared priors is part of what Arm D probes but is not otherwise controlled.
5. **Check-definition corrections** were made during analysis (leak and separation scans) and are logged; they do not touch the frozen H1 scoring rules, but a reader should audit them in the repository.

8 Conclusion

At a 24-task horizon with two requirement regime changes, an append-only executable requirement ledger roughly doubles an LLM coding agent’s end-state requirement retention over a summary baseline and cuts trap-requirement violations by 77–100%—but at $2.3\times$ the tokens and with coerced regressions in which stale tests force wrong edits. Evidence-governed retirement of invalidated tests recovers that protection at roughly half the token cost and with no rewrite-channel regressions, and is the practical recommendation. Governance is not a free lunch: the more permissively an arm could retire tests, the more old regressions returned, refuting one of our pre-registered predictions. And the arm that let the agent argue for its own tests’ removal exhibited *capture*—a mechanism-level warning, stated here as a puzzle, for any memory system (learned or governed) that lets the system under test influence what it is allowed to forget. All code, data, pre-registration, and per-iteration logs are open.

References

- [1] D. Jiang, Y. Li, G. Li, Q. Li, and B. Li. HAGE: Harnessing Agentic Memory via RL-Driven Weighted Graph Evolution. *arXiv:2605.09942*, May 2026.